# Respuestas a las Preguntas de la Guía del Scientific Workshop (4_Workshop.pdf)

Artículo: **Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes (EVAPORATE)** · PVLDB Vol. 17, No. 2 (2023)

## 1. Identificación

### 1.1. Título del artículo, autores, año y publicación

- **Título:** *Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes* (Sistema **EVAPORATE**).
- **Autores:** Simran Arora, Brandon Yang*, Sabri Eyuboglu*, Avanika Narayan, Andrew Hojel (*Stanford University*), Immanuel Trummer (*Cornell University*) y Christopher Ré (*Stanford University*).
- **Año:** 2023 (publicado en octubre/noviembre de 2023 y presentado en *VLDB 2024*).
- **Publicación:** *Proceedings of the VLDB Endowment (PVLDB)*, Vol. 17, No. 2, págs. 92–105 (ISSN: `2150-8097`). Clasificación: **Revista JCR Q1** y **Congreso CORE A***.

### 1.2. Referencia bibliográfica y DOI

- **Referencia bibliográfica (IEEE/ACM):** S. Arora, B. Yang, S. Eyuboglu, A. Narayan, A. Hojel, I. Trummer, and C. Ré, "Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes," *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 92–105, 2023, doi: 10.14778/3626292.3626294.
- **DOI oficial:** https://doi.org/10.14778/3626292.3626294
- **PDF Open Access (VLDB):** https://www.vldb.org/pvldb/vol17/p92-arora.pdf · **Código:** https://github.com/HazyResearch/evaporate

## 2. Motivación y problema

### 2.1. ¿Qué problema de investigación aborda?

Aborda el problema de **generar automáticamente vistas relacionales estructuradas (tablas SQL) a partir de Data Lakes heterogéneos** compuestos por documentos semi-estructurados y no estructurados (páginas web en HTML, informes técnicos o médicos en PDF, correos electrónicos en texto plano TXT), sin contar con datos de entrenamiento etiquetados ni esquema previo. Dada una colección de $n$ documentos $D = \{d_1, \ldots, d_n\}$, el sistema descubre automáticamente el esquema de atributos $A = \{a_1, \ldots, a_m\}$ y extrae los $n$ registros estructurados $R = \{r_1, \ldots, r_n\}$.

### 2.2. ¿Por qué es relevante?

- **El 80% de los datos empresariales son no estructurados:** Sin estructurarlos en tablas, los documentos crudos de un Data Lake no permiten ejecutar consultas analíticas SQL/OLAP ni integrarse en un Data Warehouse.
- **Fracaso del Estado del Arte ante el Trilema (Generalidad – Coste – Calidad):** Los modelos Web/DOM (`DOM-LM`, `Ceres`) exigen entrenamiento supervisado por dominio y asumen que los valores están siempre en nodos hoja del árbol HTML DOM (fracasan en PDFs, texto plano o webs complejas como Wikipedia). Los sistemas NLP abiertos (`OpenIE6`) colapsan en documentos largos y generan esquemas sin normalizar (427 relaciones distintas en un solo informe de la FDA). Y pasar cada documento por un LLM comercial tiene un coste lineal $O(n)$ prohibitivo ($5.157 por cada 10.000 documentos; $1,1 millones en Wikipedia).

## 3. Propuesta

### 3.1. ¿Cuál es la principal contribución?

1. **Nuevo compromiso (trade-off) Coste–Calidad con LLMs:** Demuestran que en vez de procesar cada documento directamente con el LLM (`Evaporate-Direct`, coste lineal $O(n)$), es drásticamente más barato y más

preciso usar el LLM para **sintetizar múltiples funciones extractoras en Python** sobre una muestra pequeña de documentos y ejecutarlas en CPU (`Evaporate-Code+`, coste sublineal $O(m \cdot k)$ independiente de $n$).

2. **El sistema `Evaporate-Code+` y el Algoritmo 1 de Supervisión Débil (Weak Supervision):** Adapta la teoría de *Weak Supervision* para funciones generadas por IA (donde el 51% de las funciones tiene <50 F1 y devuelve texto libre), aportando garantías teóricas de complejidad muestral (Proposición 1).

#### 3.2. ¿Qué método, arquitectura o técnica utilizan?

Utiliza 6 plantillas de prompt agnósticas al dominio en una arquitectura de 3 etapas (precedida opcionalmente por clustering **TF-IDF + K-Means** si hay múltiples fuentes mezcladas):

1. **Etapa 1 (Schema Synthesis):** Toma una muestra de $k = 10$ documentos, los divide en *chunks*, extrae atributos candidatos con el LLM y los re-rankea por frecuencia priorizando atributos de alta cobertura (*head attributes*).
2. **Etapa 2 (Diverse Code Synthesis):** Para cada atributo, usa dos plantillas de prompt (`P_A` orientada a expresiones regulares `re` y `P_B` con ejemplos que importan librerías como `BeautifulSoup`) sobre distintos pasajes para sintetizar un conjunto diverso de funciones Python $F = \{f_1, \ldots, f_k\}$.
3. **Etapa 3 (Function Aggregation - Algoritmo 1):** Ejecuta las funciones en CPU (0,25 ms/doc) y las evalúa sobre *D_eval* (≤10 docs) frente a la extracción directa del LLM: (1) estima la tasa de presencia $e$ del atributo para distinguir abstenciones de valores nulos reales; (2) filtra todas las funciones ruidosas con puntuación empírica $\hat{a}_j \leq 0.5$; (3) discretiza el texto libre en los *top-b buckets* más frecuentes; y (4) aprende un modelo gráfico latente para emitir el valor consensuado de cada celda.

## 4. Evaluación y resultados

#### 4.1. ¿Cómo validan su propuesta?

Validan el sistema en dos configuraciones: **ClosedIE** (esquema dado) y **OpenIE End-to-End** (descubrimiento automático de esquema + extracción), comparando `Evaporate-Code+` contra `Evaporate-Direct`, `Evaporate-Code`, `OpenIE6` y sistemas supervisados del estado del arte en web (`DOM-LM`, `ZeroShotCeres`, `RoBERTa-Structural`), junto con estudios de ablación (Tabla 4) y análisis de puntos de corte (Figura 3).

#### 4.2. ¿Con qué datos, experimentos o métricas?

- **Datos:** 16 benchmarks reales de 5 dominios y 3 formatos: *SWDE Movies* (8 webs HTML), *SWDE Universities* (5 webs HTML), *Wikipedia NBA Players* (HTML complejo), *Enron Emails* (500.000 correos TXT) y *FDA 510(k) Reviews* (informes médicos en PDF convertidos a TXT de ~20 páginas).
- **Modelos y Métricas:** 4 LLMs (`text-davinci-003`, `GPT-4`, `Claude-v1`, `Jurassic-2`) evaluados con **Text F1** (ClosedIE), **Pair F1** (OpenIE), tokens consumidos y coste en dólares ($).

#### 4.3. ¿Cuáles son los principales resultados?

- **Reducción de coste de 110× (y 378× en tamaño real):** Por cada 10.000 documentos, reduce el coste medio de $5.157 (258M tokens) en `Direct` a solo $39 (1,9M tokens) en `Code+`.
- **Mejora de calidad de +10,1 puntos Pair F1 (+18%) sobre el propio LLM directo:** Obtiene **66.7 Pair F1** medio frente a 56.6 de `Direct` y 42.8 de `Code` (+23,4 puntos en Wikipedia NBA y +17,3 puntos en informes FDA), y **82.3 Text F1** en ClosedIE.
- **Superación del SoTA supervisado sin entrenar:** Supera a `DOM-LM` y `ZeroShotCeres` en SWDE en un **+6% en OpenIE** y **+10% en ClosedIE** con cero datos de entrenamiento.

## 5. Análisis crítico

#### 5.1. ¿Qué ventajas y limitaciones presenta?

- **Ventajas:** (1) Elimina la inconsistencia y las alucinaciones del LLM directo (en FDA, `Direct` olvida el 27,5% de los atributos por documento, alucina 9,7 atributos inexistentes y renombra el mismo atributo de 4 formas distintas, mientras que el código Python logra un 78,5% de Precisión y consistencia determinista); (2) Generalidad multi-formato (HTML, PDF, TXT) y código auditable.

- **Limitaciones:** (1) En textos libres sin regularidad sintáctica (*Enron Emails*), `Direct` (93.8 F1) supera a `Code+` (86.9 F1); (2) Deja de ahorrar tokens frente a `Direct` si hay menos de $n < 40$ documentos o esquemas ultra-anchos de más de $m > 2.500$ atributos; (3) Depende de la calidad del LLM como pseudo-oráculo en los 10 documentos de *D_eval*.

### 5.2. ¿Qué preguntas quedan abiertas?

1. **Arquitectura Híbrida Adaptativa:** Usar `Evaporate-Code+` en CPU por defecto e invocar a `Evaporate-Direct` únicamente en las celdas donde las funciones Python se abstengan.
2. **Detección de *Schema / Layout Drift*:** Detectar automáticamente cuándo cambia el DOM de una web para re-sintetizar solo las funciones afectadas.
3. **Ejecución segura (Sandboxing):** Aislar el código Python generado por el LLM frente a expresiones regulares catastróficas (*ReDoS*) o código inseguro.

## 6. Relación con la asignatura

### 6.1. ¿Qué conceptos de la asignatura aparecen en el trabajo?

- **Bloque I (Almacenamiento de Datos):** Paso de datos no estructurados a accionables (`B1_1_4_Fuentres De Datos.pdf`, diap. 6 y caso FDA en diap. 14), **Web Scraping** con `BeautifulSoup`, `XPath` y `regex` (diaps. 20–32), pipelines **ETL/ELT en Data Lakes** y transformaciones **Model-driven vs. Data-driven** (`B2_2_2_Gestión De Informacion.pdf`, diaps. 9–27).
- **Bloque II (Recuperación de Información y RAG): Chunking**, búsqueda léxica, clustering **TF-IDF + K-Means** (`2_1_1` y `2_2_RAG.pdf`) y métricas **Precision, Recall y F1**.
- **Bloque III (Gobernanza e Interoperabilidad):** Descubrimiento automático de esquemas (*Schema Discovery*) y normalización de atributos (`Block 3.1.pdf` y `[3.2] Block 3.2.pdf`).

### 6.2. ¿En qué amplía o mejora lo estudiado en clase?

1. **Automatiza la programación manual de Web Scraping:** El LLM sintetiza automáticamente los parsers en `BeautifulSoup` y `regex` que en clase programábamos manualmente.
2. **Resuelve la limitación de la diapositiva 14 de ETL/ELT (*"una función rara vez cubre el 100% de los casos"*):** Combina múltiples funciones imperfectas mediante Supervisión Débil para pasar de 42.8 a 66.7 Pair F1.
3. **Une los enfoques *Data-driven* y *Model-driven*:** Utiliza un modelo *Data-driven* (LLM) para sintetizar reglas *Model-driven* (código Python) ejecutables a escala en CPU.